ECE551: Digital System Design and Synthesis

Fall 2026

Dataflow Verilog

Department of Electrical and Computer Engineering
UNIVERSITY OF WISCONSIN–MADISON

Dataflow Verilog

- Describes **how data flows**
- Continuous **assign** statements

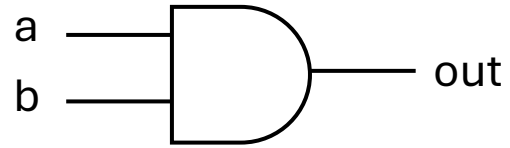
```
assign <LHS expr> = <RHS expr>;
```

  - If RHS result changes, LHS is updated with new value

```
assign [drive_strength] [delay] list_of_net_assignments;
```
Where:
```
list_of_net_assignment ::= net_assignment [{,net_assignment}]
```
& Where:
Net_assignment ::= net_lvalue = expression

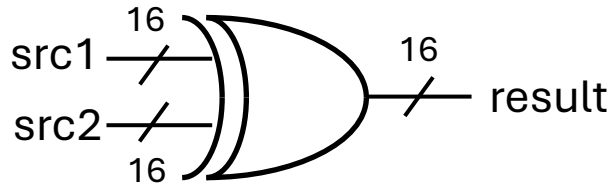
- Simplest form:

```
// out is a net, a&b are also nets
assign out = a & b;   // AND gate functionality
```



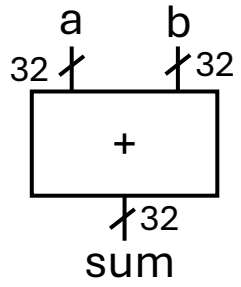
- Using vectors:

```
logic [15:0] result, src1, src2;     // 3 16-bit wide vectors
assign result = src1 ^ src2;         // 16-bit wide XOR
```



- Arithmetic operators:

```
logic [31:0] sum, a, b;       // 3 32-bit wide vectors
assign sum = a + b ;          // 32-bit adder
```



Dataflow Verilog

```
// 32-bit multiply-accumulate unit A*B+C. Sets overflow to 1 if result cannot be represented in 32 bits
module mac(output result [31:0], output overflow, input [15:0] A, B, input [31:0] C);

  assign {overflow,result} = a * b + c;

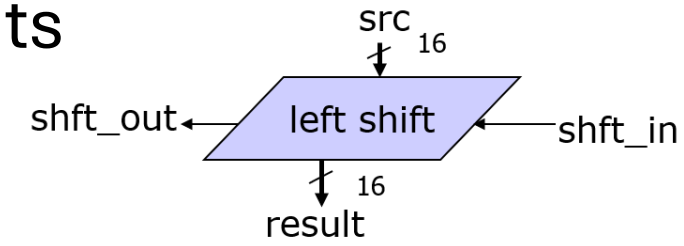
endmodule
```

- Multiple “operations” in one line!
- RHS: can use a variety of logical, arithmetic, relational operators
  - Bitwise: &, |, ^, ~
  - Logical: &&, ||, !
  - Relational: <, <=, >, >=
  - Equality: ==, ===, !=, !==
  - Conditional: (cond)? Tval:Fval;
  - Shifts: <<, >>, >>>
  - Arith: + - * / %
  - Reduction: &(sig), |(sig), ^(sig)
  - Concat: {A,B}
  - Replicate: {3{A}}
- LHS: can bundle signals together to hold the result of RHS

Concatenation

- Bundles multiple signals together as one vector
  - General form: { <sig>,<sig> [,<sig>]}
  - Example: `{2'b10, 2'11}` → `4'b1011`
- Can show up on either LHS or RHS of assignments

```
assign {shft_out,result} = {src,shft_in};
```



- Can show up in port list

```
module rippleCarryAdd8(input [7:0] A, B, output [7:0] S, output Cout);

  logic [6:0] carries;

  fulladder adders[7:0](.A(A), .B(B), .Cin({carries, 1'b0}), .Cout({Cout, carries}), .S(S));

module
```

Concatenation

- Often used to match signal widths for arithmetic operations

```
module off_corr(
  input [15:0] raw_sensor,        // raw signed 16-bit sensor reading
  input [7:0] offset,             // +/- offset to correct sensor reading
  output [15:0] corr_sensor       // corrected sensor reading
);

  assign corr_sensor = raw_sensor + offset;

endmodule
```

```
assign corr_sensor = raw_sensor +
                     {offset[7],offset[7],offset[7],offset[7],
                      offset[7],offset[7],offset[7],offset[7],offset};
```

A lot of typing… there is a better way!

Replication

- Can replicate a signal N times and concatenate together
  - General form: {N{sig}}
  - Example: `{3{2'b10}}` → `6'b101010`
    `{2'b00,{3{2'b10}},2'b11}` → `10'b0010101011`

```
assign corr_sensor = raw_sensor +
                    {offset[7],offset[7],offset[7],offset[7],
                     offset[7],offset[7],offset[7],offset[7],offset};
```



```
assign corr_sensor = raw_sensor + {{8{offset[7]}}, offset};
```

Other Handy Uses of { }

- Divide by using arithmetic shift right

```
assign signedDiv8 = {{3{value[15]}},value[15:3]};          // ASR by 3

logic [15:0] value, signedDiv8;
logic signed [15:0] value, signedDiv8;

assign signedDiv8 = (value>>>3);  // ASR operator. Need to make sure value and signedDiv8 are
                                  // declared as signed
```

- Rotate a signal

```
assign rotated = {value[0],value[15:1]}; // rotate right by 1
```

Supported Operators

- Shifts: <<, >>, >>>

```
logic [3:0] A,B,C;
logic signed [3:0] D;

assign A = 4'b1001 << 2;           // logical left shift A = 4'b0100
assign B = 4'b1001 >> 2;           // logical right shift B = 4'b0010
assign C = 4'b1001 >>> 2;          // arithmetic right shift of unsigned C = 4'b0010
assign D = $signed(4'b1001) >>> 2; // both src and dst are signed D = 4'b1110
```

- Relational: <, >, <=, >=

```
logic [3:0] A,B;
assign A = 4'b1111;
assign B = 4'b0101;

(A < B)                            // 0 (false), unsigned comparison
($signed(A) < $signed(B))          // 1 (true), signed comparison since both operands signed
($signed(A) < B)                   // 0 (false), unsigned comparison if mixed
(A > 4'b01X1))                     // X (unknown)
```

Supported Operators

- Equality: ==, !=, ===, !==

```
logic [3:0] A,B;
assign A = 4'b1001;
assign B = 4'b10x1;

(A == B)        // X (unknown), x could be 0 or 1
(A === B)       // 0 (false), x not equal to 0
```

- Logical: &&, ||, !
- Bitwise: &, |, ^, ~

```
logic [3:0] A,B,C,D;
assign A = ~(4'b0011);   // bit-by-bit, 4'b1100
assign B = !(4'b0011);   // logical, 4'b0000

assign C = 4'b0011 &  4'b1110;   // bit-by-bit, 4'b0010
assign D = 4'b0011 && 4'b1110;   // logical, 4'b0001
```

Supported Operators

- Reduction: &(sig), |(sig), ^(sig)

```
logic [3:0] A;
assign A = 4'b1101;

&(A)              // all bits set? 0
|(A[2:0])         // some bits set? 1
^(A)              // odd parity? 1
```

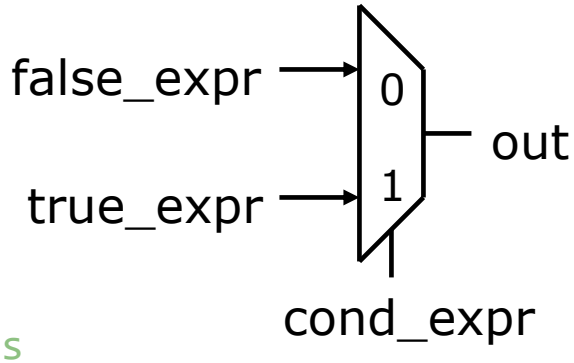
- Arithmetic : +, -, *, /, %
  - ** (exponent) not synthesizable
  - Pay attention to bitwidths

```
logic [7:0] prod;
logic [15:0] quotient;

assign prod = a[3:0] * b[3:0];
assign quotient = a[15:0] / b[7:0];
```

Supported Operators

- Conditional
  - General form: cond_expr? true_expr : false_expr
  - Can be nested




```
assign out = select?in1:in0;                              // a 2-to-1 mux
assign src1 = rf2src1?mem[addr1]:16'bhzzzz;                // tri-state bus
assign out = sel[1]?(sel[0]?in3:in2):(sel[0]?in1:in0);     // a 4-to-1 mux
```

```
localparam add    = 3'b000;
localparam and    = 3'b001;
localparam xor    = 3'b010;
localparam shft_l = 3'b011;
localparam shft_r = 3'b100;

// an ALU capable of arithmetic, logical, shift, and zero
assign {cout,dst} = (op==add)    ? src1+src2+cin :
                    (op==and)    ? {1'b0,src1 & src2} :
                    (op==xor)    ? {1'b0,src1 ^ src2} :
                    (op==shft_l) ? {src1,cin} :
                    (op==shft_r) ? {src1[0],src1[15],src1[15:1]} :
                                   17'h00000;
```

This can be very confusing to read if not coded with proper formatting…

Saturation Example

- Assume we have a 16-bit signed **sum** and want to saturate it to a 12-bit signed value for use in downstream calculations.
  - `if (sum > 16'h07FF) → 12'h7FF`
    `else if (sum < 16'hF800) → 12'h800`
    `else → sum[11:0]`

```
// use relational operators
assign sat12 = (sum > $signed(16'h07FF)) ? 12'h7FF :
               (sum < $signed(16'hF800)) ? 12'h800 :
               sum[11:0];

// use reduction and logical operators
assign sat12 = (!sum[15] && |sum[14:11])   ? 12'h7FF :
               (sum[15] && !(&sum[14:11])) ? 12'h800 :
               sum[11:0];

// use reduction and bitwise operators
assign sat12 = (~sum[15] & |sum[14:11]) ? 12'h7FF :
               (sum[15] & ~&sum[14:11]) ? 12'h800 :
               sum[11:0];
```

Smaller faster circuit

Arrays & Memories

- Vectors: `logic [3:0] A;  // 4-bit wide, packed bits`
- Array of vectors:

```
logic [3:0] array [0:9];          // 10-element array of 4-bit vectors, unpacked elements
logic [3:0] array [9:0];          // legal syntax...
logic [7:0] array2d [0:99][0:4]; // 2D array (100x5) of 8-bit vectors
```

- Often have to model memories

```
module RF(
  input [4:0] addr1, addr2,      // read and write addresses
  input rd,wrt,                  // read and write enables
  input [15:0] wrt_data,         // write data
  output [15:0] src1,src2        // source busses to ALU
);

logic [15:0] reg_file[0:31];     // Register file of 32 16-bit words
...
assign src1 = (rd) ? reg_file[addr1] :   // transfer addressed register
                     16'hzzzz;           // otherwise back off bus
...
```

